PROGRAMS, CIRCUITS, STATE, AND GRADIENTS

Handwired, from the ground up

A learning notebook for understanding the machinery—and finding an experiment worth doing.

You do not need to believe that this approach is useful before learning it. The aim is to understand exactly what is being represented, what runs, what can be differentiated, and which claims still need evidence.

This guide assumes basic JavaScript: variables, functions, arrays, and an `if` statement. It develops the mathematical notation as it goes. No previous neural-network or calculus knowledge is required.

> **The central mental model**
>
> Handwired is a toolkit for constructing a fixed numeric computation graph. A host repeatedly feeds it numbers, evaluates it, saves selected results as state, and uses other results to update an application. Some graphs implement hand-written rules; some contain adjustable parameters that can be trained.

How to use this notebook

Read sections 1–7 in order. Calculate the small examples before running them. Sections 8–10 explain gradients and the demos; section 11 proposes experiments with explicit baselines and failure criteria. Section 12 gives exercises and answers.

Scope and provenance. This describes the checked-out repository, not an abstract promise about arbitrary JavaScript. The source map at the end names the implementations behind the explanation. Equations initially use real arithmetic; section 6 explains where actual FP32 execution differs.

The route

1 Start with an ordinary calculation 3
2 The building blocks: wires and operations 4
3 Two times: construction and execution 5
4 How arithmetic implements decisions 6
5 State is feedback managed by the host 7
6 What actually executes 8
7 Build and run a switch 9
8 Gradients, without mysticism 10
9 An actual learning step—and its limits 11
10 Read the demos with the right questions 12
11 A route toward an original contribution 13
12 Exercises, answers, and source map 15

> **Three distinctions to keep throughout**
>
> **Building is not evaluating.** Builder JavaScript constructs a graph; the evaluator later runs it.
>
> **Evaluating is not training.** A fixed-weight graph can run forever without learning anything.
>
> **Having a backward pass is not having a useful learning signal.** Hard decisions, saturation, and missing paths can stop useful gradients.

1 Start with an ordinary calculation

Suppose a window is 100 pixels wide and should be centered in a viewport of width v. Its left coordinate is

$$x = 0.5v - 50.$$

For $v = 800$, the answer is 350. This is just arithmetic. There is no learning, probability, or inference about what a window means.

Now break the expression into a tiny instruction: "read the viewport width, multiply it by 0.5, add −50, and store the result." That is an **affine operation**: a weighted sum plus a constant.

$$y = b + w_1x_1 + w_2x_2 + ... + w_nx_n.$$

The x_i are input numbers, the w_i are coefficients (often called weights), and b is the bias. A bias is simply the constant part. A weight is not necessarily something learned: 0.5 is a weight because centering requires one half.

1.1 From one result to several

A window might calculate its left coordinate and width together. This time, let its width be an input u rather than a fixed 100:

$$\begin{pmatrix} x \\ w \end{pmatrix} = \begin{pmatrix} 0.5 & -0.5 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} v \\ u \end{pmatrix} + \begin{pmatrix} 0 \\ 0 \end{pmatrix}.$$

Read a matrix row as one affine operation: multiply corresponding entries and add. The first row says $x = 0.5v - 0.5u$; the second says $w = u$. At $v = 800$ and $u = 100$, the outputs are 350 and 100.

There is nothing mystical about a matrix multiplication; it batches several weighted sums. The coefficients express relationships we already chose. They do not need to be discovered by training.

1.2 Why addition alone is not enough

Stacking affine operations still produces an affine result. For example, replacing x in $y = 3x + 1$ with $x = 2v - 4$ gives $y = 6v - 11$. It cannot express a kink, threshold, or choice.

Handwired adds a nonlinear operation:

$$R(x) = \max(0, x).$$

This is the **rectified linear unit**, or ReLU. It keeps positive values and turns negative values into zero. At $x = -3, 0, 4$, its outputs are $0, 0, 4$.

A useful construction follows immediately:

$$\text{clamp}(x, 0, 1) = R(x) - R(x - 1).$$

Below zero both terms vanish. Between zero and one only the first grows. Above one both grow at the same rate, leaving a difference of one. This is a real-arithmetic identity; floating-point implementations need care at large magnitudes.

> **Checkpoint**
>
> What is learned when the program uses a weight of 0.5 to center a window? **Nothing.** The coefficient encodes a rule chosen by the developer.

2 The building blocks: wires and operations

2.1 A wire is an address, not a current value

An evaluation fills an array of numbers. First come the **source cells**: inputs, constants, and old state. Operation results follow them. A **wire** identifies one position in this conceptual array.

If a wire has address 7, it does not mean the program has calculated the number 7. It means "read the number stored at position 7." Handwired gives wires a distinct TypeScript type to help prevent this confusion.

For a conceptual two-input graph:

Position	Meaning	For this evaluation
0	Input a	3
1	Input b	4
2	$s = a + b$	7
3	$p = ab$	12
4	$y = s + p$	19

This is a teaching graph, not a promise about the compiler's final wire numbering. The compiler may combine small sums, remove unused work, and pack values differently.

2.2 The arithmetic core

An **affine** instruction computes a weighted sum. A **ReGLU** instruction computes

$$y = \ell \times R(g),$$

where both ℓ and g are affine forms. "ReGLU" means a rectified linear gate multiplying another value. With $\ell = 1$, it becomes an ordinary ReLU.

With a nonnegative b, the product ab is simply $aR(b)$. For a signed b, use

$$ab = aR(b) - aR(-b).$$

Thus ordinary multiplication can be constructed from the same ingredients. The actual `circuit.multiply` builder chooses a shorter form when the declared ranges prove an operand nonnegative, and records the whole product for its backward pass.

2.3 The current runtime has four instruction kinds

The repository's slogan emphasizes weighted sums and ReGLU, but the current implementation also has two explicit instructions. Do not force every implementation detail into the slogan.

Instruction	Role
`affine`	A weighted sum plus a bias.
`reglu`	One affine form times the ReLU of another.
`boolean`	Round a score in 0–1 to an exact bit, with a threshold at 0.5. It deliberately relies on specified FP32 rounding.
`index`	Read an earlier value from a fixed candidate table using an integer index; declared out-of-range rules either clamp or return zero.

Higher-level helpers such as `emitCompare`, `emitSelect`, and `emitAnimatedObject` are **builders**, also called macros. They append lower-level operations. They are not extra magic performed by a model.

For the boolean instruction, an idealized real-arithmetic simplification would give the wrong answer: its implementation uses the rounding of `(score + K) - K`, with $K = 8_{388_{609}}$, to produce a bit. An index read also has explicit lookup semantics. These details matter when designing another backend.

3 Two times: construction and execution

3.1 Builder JavaScript constructs the graph

This is a genuine Handwired expression:

```
const left = circuit.affine([[viewportWidth, 0.5]], -50)
```

`viewportWidth` is a wire. The inner pair means "read this wire with coefficient 0.5." The last argument is the bias. `left` is another wire, not the current coordinate.

This line runs while building. Later evaluations can use $v = 800$, then $v = 1000$, without rebuilding. The same graph returns 350, then 450.

By contrast, this is generally a mistake:

```
if (viewportWidth > 800) {
  // This compares the wire's numeric address while building.
  // It is not a runtime condition on the viewport's value.
}
```

The TypeScript distinction helps with many argument mistakes, but does not turn JavaScript operators into graph operations. There is no general source-to-source conversion of arbitrary JavaScript here.

3.2 A complete graph you can run

The following is **Program A**. Each complete program in this notebook is independent. To run one, save its block as a temporary `.ts` file in the repository root and run `bun <filename>.ts`. Imports assume that location. Shorter blocks elsewhere are illustrative fragments.

```
import {
  createSourceTable, createEvaluator, input,
} from './src/runtime/cells.ts'

const table = createSourceTable()
const cells = table.declare({
  viewportWidth: input([100, 2000], {initial: 800, grid: 1}),
})
const circuit = table.createCircuit()
const left = circuit.affine([[cells.viewportWidth, 0.5]], -50)
const built = {cells, output: {left}, ...table.finish()}
const evaluate = createEvaluator(built)
const source = new Float32Array(built.initialSource)

for (const width of [800, 1000]) {
  source[cells.viewportWidth] = width
  const {output} = evaluate(source)
  if (output.left !== width / 2 - 50) throw Error('wrong position')
  console.log(width, output.left)
}
```

Expected pairs: `800 350` and `1000 450`.

`table.declare` assigns source addresses. `table.createCircuit` installs the declarations before operations are built. `table.finish` produces the initial table and the next-source mapping. `createEvaluator` compiles the requested outputs and next values, and returns a validating evaluation function.

The loop above is ordinary **host** JavaScript running two evaluations. A loop in builder code with a fixed iteration count would instead append repeated pieces of graph. These are different jobs.

4 How arithmetic implements decisions

4.1 Booleans as exact bits

Represent false as 0 and true as 1. On this restricted domain:

$$\mathrm{NOT}(b) = 1 - b, \quad \mathrm{AND}(a, b) = R(a + b - 1),$$

$$\mathrm{OR}(a, b) = a + b - R(a + b - 1).$$

Check all four input pairs. For example, AND returns 1 only for $(1, 1)$, because only then is $a + b - 1$ positive.

These formulas do not make arbitrary numbers into booleans. At $(0.7, 0.7)$, AND returns 0.4. "These inputs are exact bits" is a required contract.

4.2 A branch becomes selection

If b is an exact bit, a choice can be written

$$y = (1 - b)y_0 + by_1.$$

At $b = 0$, choose y_0; at $b = 1$, choose y_1. Handwired's call order is `emitSelect(circuit, bit, off, on)`.

Unlike an ordinary JavaScript `if`, both branch calculations are present in the graph and run when they are live dependencies of the requested outputs. Selection is not short-circuit evaluation. An invalid unused branch can still poison the result: infinity times zero is NaN, not zero.

4.3 Where does a comparison bit come from?

For integer a within safe numeric bounds, consider

$$b = R(a - 4) - R(a - 5).$$

a	$R(a-4)$	$R(a-5)$	b
3	0	0	0
4	0	0	0
5	1	0	1
6	2	1	1

For integer inputs, this is the bit for $a > 4$. But at $a = 4.5$ it returns 0.5. It is a ramp between adjacent integers, not a universal exact comparison of arbitrary real numbers.

The runtime's `emitCompare` handles the actual contracts. It requires finite declared ranges. For suitable grid-aligned operands it uses compact integer-bit constructions; otherwise it builds a longer sign cascade to resolve representable FP32 differences. One construction for integer D is

$$[D \geq 0] = R(1 - R(-D)).$$

Square brackets here mean "1 when the proposition is true, otherwise 0." For $D = -1, 0, 1$, the results are $0, 1, 1$.

4.4 From a comparison to a hit test

For a rectangle with left l, top t, width w, and height h:

$$\text{inside} = \text{AND}\big([p_x \geq l], [p_x < l + w], [p_y \geq t], [p_y < t + h]\big).$$

The four-input AND is $R(b_1 + b_2 + b_3 + b_4 - 3)$. Once you can compare, combine bits, and select numbers, you can express much of an interface's event logic. Exact hit-test policies differ between demos; the formula here uses a simple half-open rectangle.

5 State is feedback managed by the host

A graph only reads earlier values. How can it remember that a switch was turned on?

Separate **old state** from **next state**:

$$(s_{t+1}, o_t) = F(s_t, i_t).$$

F is one evaluation, s_t is stored state, i_t is the current input, and o_t is output to the application. After evaluation, the host copies the next-state numbers into the source cells for a later evaluation.



Host saves next state for the following evaluation.

This is recurrence over time, not a cycle inside one evaluation. A graph can therefore describe a state machine without mutating its own intermediate wires.

5.1 The smallest stateful example: a flip event

Let s be the switch bit and e a flip event bit:

$$s' = (1 - e)s + e(1 - s).$$

Old s	Event e	Next s'
0	0	0
0	1	1
1	0	1
1	1	0

A click is normally a more involved interaction: press arms a control, release inside activates it, release elsewhere or cancel disarms it. This tiny example starts after the host has already decided that a flip event occurred. Section 7 connects it to the real template.

5.2 Cell policies

Declaration	What happens across evaluations
`input` / `inputBit`	The host writes it; the graph preserves its value in the next-source mapping.
`constant` cell	Initialized once and preserved; useful for externally tuned parameters. "Constant" is a state policy, not an optimizer's promise never to update it.
`event`	The host raises an exact bit; its next value is zero. The host must commit next values to clear it.
`state` / `stateBit`	The graph calculates its next value, registered with `table.next`.

Each state cell needs exactly one next-value assignment. `table.next` registers a relationship while building; it does not immediately change stored state.

A constant embedded in an operation is different from a constant cell. `circuit.constant(20)` may fold into a bias, so changing it generally requires rebuilding. A source cell can change without changing graph structure.

6 What actually executes

6.1 A compiled dependency graph, packed into layers

An operation depends on its input wires. Independent operations can share a layer; dependent ones must come later. For the teaching graph $s = a + b$, $p = ab$, $y = s + p$, the sum and product are independent, and the last addition depends on both. Multiplication may itself expand into more primitive operations, so this conceptual graph is not a literal primitive-layer count.

Compilation starts from requested output wires, called **roots**. It removes operations that cannot reach those roots, packs live values, and collects affine forms into sparse matrix projections. ReGLU gates, boolean rounding, and lookup instructions complete the layers' work.

A **sparse matrix** stores the nonzero coefficients rather than every possible connection. Most program values depend on a few earlier values, not every value, so this representation matters.

A neural-network analogy is useful here: a fixed graph, weighted connections, layers, and nonlinearities. It does not imply learned intelligence, dense layers, probabilistic output, or GPU execution. The normal runtime and the desktop demo execute in JavaScript on the CPU. The JSON-search demo has its own WebGPU backend.

6.2 Exact about what?

The evaluator uses 32-bit floating-point numbers, or FP32. JavaScript ordinarily performs arithmetic in 64-bit floating point. The conceptual affine rule $b + \sum_i w_i x_i$ is implemented with rounding of each coefficient/product/accumulated result, in stored order.

Consequences:

- Decimal fractions such as 0.1 generally have approximate representations.
- Reordering a sum or fusing operations can change its result.
- FP32 represents every integer between -2^{24} and 2^{24}, but not every larger integer.
- Exact comparisons concern the operands' FP32 values within the declared contracts—not arbitrary original real numbers before conversion.

A range declaration is a promise, not a clamp. A grid declaration says values are multiples of a specified power of two, such as 1/64 pixel. It does not itself round input to that grid. The host must do the rounding, and validation can catch violations.

These declarations can enable shorter comparisons and indexed reads. They are not optional decorative type hints: false promises can invalidate an optimization. The high-level evaluator checks source and next-state cells; lower-level code may need explicit validators for computed grids, indices, and one-hot groups.

6.3 Loops, work, and cost

A builder loop over 12 photos can append 12 sets of calculations. A runtime "keep going until done" loop cannot simply appear inside this fixed graph. Use a fixed bound, restructure the computation, or perform repeated evaluations in the host.

There are two different kinds of repetition to measure: work inside one evaluation, and how often the host evaluates. In particular, both branches of a selection may cost work even when one is not chosen. Arithmetic packaging does not automatically beat conventional branchy code.

The Monitor's source-cell count, live operations, layer count, and stored weights describe this numeric program. A stored weight is a sparse projection coefficient, not necessarily a learned model parameter. Compiled output buffers can be reused and overwritten by the next call; copy results if retaining a history.

7 Build and run a switch

This is **Program B**: the flip-event state machine, using the real cell and evaluator APIs. It intentionally omits pointer hit tests and animation so you can see every state transition.

```
import {
  createSourceTable, createEvaluator, event, stateBit,
} from './src/runtime/cells.ts'
import {emitSelect} from './src/runtime/numeric-ui-core.ts'

const table = createSourceTable()
const cells = table.declare({flip: event(), on: stateBit()})
const circuit = table.createCircuit()
const inverted = circuit.affine([[cells.on, -1]], 1)
const nextOn = emitSelect(circuit, cells.flip, cells.on, inverted)
table.next(cells.on, nextOn)
const built = {cells, output: {on: nextOn}, ...table.finish()}
const evaluate = createEvaluator(built)
const source = new Float32Array(built.initialSource)

const events = [0, 1, 0, 1]
const expected = [0, 1, 1, 0]
for (let step = 0; step < events.length; step++) {
  source[cells.flip] = events[step]!
  const result = evaluate(source)
  if (result.output.on !== expected[step]) throw Error('wrong state')
  source.set(result.nextValues)
  if (source[cells.flip] !== 0) throw Error('event was not cleared')
  console.log(step, result.output.on)
}
```

The printed states are $0, 1, 1, 0$. The important line is `source.set(result.nextValues)`. Without a commit, the evaluator has produced next state, but no one has stored it for the following call.

7.1 Add one ingredient at a time

To reach the real switch in `template/toggle.ts`:

1. Add input cells for pointer coordinates and event bits for down, up, and cancel.
2. Build a rectangle hit test from four comparisons.
3. Add an `armed` state bit. Press inside arms; release inside while armed flips; release or cancel disarms.
4. Map the on bit to a thumb destination: $d = 20s$ for 20 pixels of travel.
5. Animate the thumb toward that destination and expose its current position as output.
6. Let the browser draw that output with CSS transforms and accessibility attributes.

The Notes window in the desktop still contains a browser-managed text field. Rendering, text editing, event delivery, and clock scheduling are not secretly performed by affine operations.

7.2 Why springs fit the representation

A simple spring step can update displacement q and velocity v by a 2-by-2 matrix:

$$\begin{pmatrix} q' \\ v' \end{pmatrix} = \begin{pmatrix} m_{11} & m_{12} \\ m_{21} & m_{22} \end{pmatrix} \begin{pmatrix} q \\ v \end{pmatrix}.$$

If one fixed-duration step is matrix M, then n steps are M^n. The host calculates the appropriate matrix power and supplies four matrix entries; the graph applies that matrix once. The template uses fixed 4 ms steps and schedules time advancement before handling events at their timestamps.

This is an example of **reformulating an algorithm**, not blindly translating a loop. It also marks a boundary for differentiation: the graph can differentiate with respect to matrix entries, but stiffness and damping used by host code to generate those entries are outside that graph. Learning them requires completing the derivative chain through that host computation or expressing it differentiably elsewhere.

8 Gradients, without mysticism

8.1 A derivative is a local sensitivity

For $y = 3x$, increasing x by a small amount ε increases y by 3ε. Its derivative is 3: the output's change per unit change of input, locally.

For $y = x^2$, expand $(x + \varepsilon)^2 = x^2 + 2x\varepsilon + \varepsilon^2$. For very small ε, the leading change is $2x\varepsilon$, so the derivative is $2x$.

A **gradient** is the collection of these sensitivities with respect to several inputs or parameters. It answers "which small parameter changes would change this scalar result, and how strongly?" It is not a plan, an explanation of intent, or a proof that a useful solution exists.

8.2 The chain rule connects operations

Let $z = wx$, and let the error against target t be

$$L = \frac{1}{2}(z - t)^2.$$

L is a **loss**: a scalar describing mismatch. Its derivative with respect to z is $z - t$. The derivative of z with respect to w is x. Multiply along the path:

$$\frac{\partial L}{\partial w} = (z - t)x.$$

At $x = 2$, $w = 1$, and $t = 6$, the prediction is 2, loss is 8, and gradient is −8. A gradient-descent step with learning rate $\eta = 0.1$ is

$$w_{\text{new}} = w - \eta \frac{\partial L}{\partial w} = 1 - 0.1(-8) = 1.8.$$

The new prediction is 3.6 and the loss is 2.88. One step reduced error. A step can also overshoot or fail; the learning rate and objective matter.

8.3 Backpropagation is efficient bookkeeping

Run the graph forward and keep intermediate values. Start with the loss's sensitivity to itself, which is 1. Walk operations backward, distributing sensitivity to their inputs using each operation's derivative. Add contributions when several paths reach the same input.

For an affine operation, each input receives the output gradient times its coefficient. For $y = ab$, the contributions are b times the output gradient to a, and a times it to b. For a ReLU, the derivative is 1 on the positive side and 0 on the negative side; this implementation chooses 0 at the corner.

Handwired records multiplication and selection macros so their backward rules can follow the intended whole expressions, rather than inheriting an unhelpful zero from a particular ReLU decomposition at a corner.

The current `backpropagateReference` returns gradients with respect to wires, including source cells. To train a parameter without rebuilding, put it in a source cell and make its use a graph operation. A coefficient embedded in `affine` is not automatically exposed as an independently trainable source parameter.

8.4 What "freeze part of the program" means

Suppose $y = F\left(G_{\theta(x)}\right)$. G_θ has adjustable parameters θ; F is a fixed calculation. Backpropagation can travel through F into G_θ even though no parameter of F is updated. Fixed does not mean "blocks the gradient."

In practice you choose which source cells the optimizer writes. Inputs, state, and trainable parameters are all numbers, but they have different meanings and update policies. Do not let an optimizer modify arbitrary state bits or a declaration's required invariants.

> **An important limitation**
>
> The backward pass differentiates the intended real arithmetic, not literal FP32 rounding. It also does not automatically differentiate through repeated host calls and state commits. Learning from a sequence requires preserving the chain across time, such as by unrolling transitions and propagating through them.

9 An actual learning step—and its limits

This is **Program C**. It reproduces the previous calculation using Handwired's backward pass. The parameter w is a source cell, not an embedded affine coefficient.

```
import {Circuit} from './src/runtime/numeric-ui-core.ts'

const circuit = new Circuit(3)
const x = circuit.source(0)
const w = circuit.source(1)
const target = circuit.source(2)
circuit.assumeRange(x, -10, 10)
circuit.assumeRange(w, -10, 10)
circuit.assumeRange(target, -10, 10)
const prediction = circuit.multiply(w, x)
const error = circuit.affine([[prediction, 1], [target, -1]])
const squared = circuit.multiply(error, error)
const loss = circuit.affine([[squared, 0.5]])
const source = new Float32Array([2, 1, 6])
const values = circuit.evaluateReference(source)

// Seed d(loss)/d(loss) = 1; all other seed gradients are zero.
const seeds = new Float32Array(values.length)
seeds[loss] = 1
const gradients = circuit.backpropagateReference(values, seeds)
if (values[loss] !== 8 || gradients[w] !== -8) {
  throw Error('unexpected loss or parameter gradient')
}
source[w] = source[w]! - 0.1 * gradients[w]!
const updated = circuit.evaluateReference(source)
if (!(updated[loss]! < values[loss]!)) throw Error('loss did not fall')
console.log('before', values[loss], 'gradient', gradients[w])
console.log('after', updated[loss]!.toFixed(2))
```

Expected output is `before 8 gradient −8`, then `after 2.88`. This uses the reference evaluator so every original wire's forward value is available to the reference backward pass. The optimizer here is the single subtraction in host code. Repeating that subtraction would train the same graph; it would not build new code.

This one-example fit learns only a coefficient that matches one target. It does not demonstrate generalization, architecture discovery, or an advantage over solving $w = \frac{t}{x}$ directly. It is a microscope for understanding the mechanism.

9.1 Three different gradient problems

A hard threshold. Let $y = [x > 4]$. For $x = 2$ and every sufficiently small change to x, y stays zero. There is no local slope pointing toward 4. Merely expressing the decision with arithmetic does not fix this.

A saturated continuous function. A clamp has zero slope outside its active interval. An optimizer may receive no signal even though the function contains no explicit boolean instruction.

A disconnected parameter. If a spring matrix was computed by host JavaScript, gradients of the graph with respect to its entries do not include the missing derivatives of those entries with respect to stiffness. No mathematical connection is created by calling two pieces of code in succession.

9.2 Exact behavior versus useful training

The runtime rejects backpropagation through a reached boolean atom by default. A caller may request a **straight-through** gradient: use the hard result in the forward pass, but pass back a surrogate slope. This is a deliberate training approximation, not the derivative of the actual hard decision.

For index reads, the default also rejects a nonzero backward signal. Opting into `indexGradient: 'zero'` allows the selected candidate to receive its gradient while giving the index form none. The alternative `'backward-difference'` uses the difference between neighboring table values as an index training signal. It is not an ordinary derivative of integer selection.

Comparisons built from ordinary ReLU instructions do not necessarily throw; a backward pass can simply return zero. Also, at corners a particular arithmetic extension can produce a convention-dependent signal. A nonzero number is not proof that gradient descent can meaningfully learn a discrete rule.

Softening changes the problem. Replacing a hard threshold with a smooth transition can make training easier, but execution now uses fractional decisions unless you restore a hard rule. Always evaluate the final hard behavior separately. A differentiable penalty is likewise not a proof of a hard constraint.

10 Read the demos with the right questions

10.1 The desktop: fixed rules in numeric form

The desktop combines window management, spring animation, photo behavior, and the Monitor's virtualized list into one numeric program. The host supplies browser events and timing inputs, evaluates, and writes results to the DOM.

The Monitor is inspecting that program. "State" shows source cells; "Operations" shows live intermediate calculations; "Outputs" shows results. The weights were deliberately constructed. No learning is necessary for dragging a window or selecting a photo.

Ask: **Which state cell remembers this action? Which operations compute its next value? What stays in host JavaScript?** These are more revealing questions than "where is the intelligence?"

10.2 The learned gallery: fit the current sketch

The separate gallery has a small network whose coefficients are stored in source cells. Fixed features of each slot's position feed two ReGLU layers and then four outputs: left, top, width, and height. The implementation has 20 output slots, with the page using up to 12 photos, and 244 coefficient cells.

Your sketch supplies target rectangles. The host computes loss and output-gradient seeds from the produced rectangles. Backpropagation sends those seeds through the layout network, and Adam updates its coefficient cells. The loss calculation and optimizer do not need to be inside the same circuit to use its backward pass.

The fitting objective combines distance from the sketch with a preference for nearby corresponding edges to align. There is additional training bookkeeping, including alignment multipliers and learning-rate adjustments. It is not just a bare neural net autonomously discovering layout rules.

Training continues as you edit. The network fits the present sketch; the demo does not establish a reusable layout policy over arbitrary new content or viewport sizes.

10.3 Why copying rectangles is a valid baseline

If the only objective is to place a photo exactly in a supplied rectangle, assigning its coordinates directly wins on simplicity and precision. If alignment is the goal, a conventional constraint solver or direct optimization over rectangle coordinates is also a baseline.

The learning demo demonstrates that this representation supports parameter fitting. It does not by itself show that a network is needed, that a new learning algorithm was invented, or that this is faster than ordinary UI code.

> **What to retain**
>
> The common representation can hold both prescribed arithmetic and trainable arithmetic. Whether that combination is useful is a question for a particular task and comparison—not a consequence of calling it a neural network.

11 A route toward an original contribution

Start with a narrow hypothesis. “Make UI differentiable” is too broad, and differentiable programming already exists. A research contribution could instead be a new exact lowering, a better compiler optimization, a useful mixed discrete/continuous training method, or a demonstrated application advantage.

The projects below are **proposals**, not claims of novelty or measured benefits. Search existing work before describing a result as new. An existing mechanism plus a new example is not necessarily a new technique.

11.1 Project A: do declarations buy real performance?

Question. Can range, grid, and one-hot information produce substantially smaller or faster programs while preserving behavior?

Small experiment. Build a repeated hit-test workload with a declared coordinate domain. Compare legal FP32 inputs with and without a grid declaration, and compare ordinary JavaScript with the compiled circuit. Include points exactly on rectangle edges and immediately outside them.

Measure. Build time, live operation count, layers, stored weights, steady-state latency, and validation overhead separately. Sweep the workload size rather than reporting one favorable case. If adding a GPU backend, include transfer and dispatch costs.

Next step. Look for a missing optimization supported by an invariant already present in the input program. Document its numeric preconditions, implement it, and compare against both the old compiler and the conventional baseline.

Disproof. Fewer operations but no end-to-end speedup is not a performance win. This repo already exploits several such declarations, so merely reproducing grid comparisons is a learning exercise.

11.2 Project B: fit shared rules, not individual rectangles

Question. Does a compact parameterized layout rule recover useful relationships from examples across viewport widths?

Small experiment. Define a deliberately restricted rule family: shared margin, shared gap, and continuous column proportions. Supply target layouts at several widths. Fit the shared parameters through geometry calculations; then evaluate widths and content sizes excluded from fitting.

Baselines. Direct parameter search, an ordinary autodiff implementation of the same formulas, and a hand-written responsive layout. A large network that memorizes each training width is not evidence of learning shared rules.

Measure. Held-out edge error, overflow, parameter count, fit time, and sensitivity to a changed example. Keep column count fixed at first; otherwise you have introduced a discrete model-selection problem.

Possible contribution. A new representation or inference method that recovers interpretable rules, or a workflow that demonstrably reduces the work needed to express them. The novelty would need to be established against inverse-layout and program-synthesis work.

Disproof. If direct search is simpler and just as good, differentiation has not earned its complexity. If parameters are unidentifiable—several settings explain the same examples—gather another example rather than declaring the fitted values "the true rules."

11.3 Project C: make the hard/soft boundary explicit

Question. Can a mixed program preserve exact discrete execution while exposing a useful, clearly specified training surrogate?

Small experiment. Use one threshold followed by two continuous branches. Fit a parameter whose effect must cross the threshold. Compare zero-gradient behavior, a softened forward decision, a straight-through backward rule, and direct search over the threshold.

Measure. Success across initializations, loss under the final hard decision, boundary errors, and how often the surrogate points in a direction that actually worsens the hard objective. Plotting surrogate loss alone would hide the core failure mode.

Possible contribution. A principled interface that states exact forward contracts and approximate backward contracts separately, or a better method for a narrowly defined class of programs. Straight-through estimation itself is established, not a new discovery.

Disproof. Lower training loss with worse hard behavior is not progress. Never weaken a required invariant just to keep a gradient alive without labeling that change.

11.4 The experiment card

Before implementing an ambitious feature, fill in:

1. **Task:** what input should produce what observable output?
2. **Unknown:** which numbers or structures are actually being inferred?
3. **Representation:** which parts are graph operations, host code, and discrete choices?
4. **Baseline:** what is the simplest non-Handwired solution?
5. **Claim:** what would be better—correctness, effort, latency, memory, or learning quality?
6. **Evidence:** which workloads and withheld cases test that claim?
7. **Failure criterion:** what result would make you abandon or revise the approach?

Do not begin by unfreezing every weight of a working UI. Many coefficients encode logic or numeric invariants; changing them can destroy correctness before producing an interpretable experiment.

12 Exercises, answers, and source map

12.1 Work these without the runtime first

1. **Affine form.** Write the graph formula for a window centered at width v with variable width u. Evaluate at $v = 1000$, $u = 240$.
2. **Bits.** Evaluate $R(a + b - 1)$ for all four exact-bit input pairs. What does it return at $a = b = 0.75$, and why does that matter?
3. **Comparison.** Evaluate $R(a - 4) - R(a - 5)$ at $a = 4$, 4.25, and 5. Which input contract makes it a comparison bit?
4. **State.** Start with on = 0 and process flip events 1, 1, 0, 1, committing every evaluation. What happens if no next state is ever committed?
5. **Gradient.** With $x = 3$, $w = 1$, and target $t = 6$, calculate loss, parameter gradient, and the next parameter for learning rate 0.1.
6. **Missing path.** The host computes $m = 2k$, writes m into a source cell, and the graph computes $y = mx$. Its backward pass reports sensitivity to m. What is needed to obtain sensitivity to k?
7. **Research judgment.** A learned gallery matches every training rectangle perfectly. What evidence is still missing for a claim that it learned a useful responsive layout rule?

12.2 Answers and what they check

1. $x = 0.5v - 0.5u = 380$. This checks that a matrix row is a relationship, not a learned fact.
2. Outputs are 0, 0, 0, 1 for pairs $(0, 0)$, $(0, 1)$, $(1, 0)$, $(1, 1)$. At $(0.75, 0.75)$ the result is 0.5. The exact-bit contract is essential.
3. Outputs are 0, 0.25, 1. Integer inputs within appropriate numeric bounds make this a bit. A fractional ramp does not implement a hard comparison.
4. Committed states are 1, 0, 0, 1. Without commits the old on bit stays 0, so outputs for those inputs are 1, 1, 0, 1; the event cell is not automatically cleared either. This distinguishes calculating state from storing it.
5. Prediction is 3, loss is 4.5, gradient is $(3 - 6)3 = -9$, and the next parameter is 1.9. The updated prediction is 5.7 and loss is 0.045 in real arithmetic.
6. Complete the chain rule outside the graph: multiply sensitivity to m by 2. Alternatively include the computation of m from k inside a differentiable graph.
7. Test unseen widths and content, constrain what the model may memorize, compare against ordinary responsive rules and direct fitting, and measure fitting cost. Training fit alone cannot establish generalization or a practical advantage.

12.3 A short glossary

Word	Meaning here
Circuit	The graph being constructed, plus its numeric declarations and builder methods.
Source cell	An input slot in the `Float32Array`: input, parameter, event, or state.
Wire	An address of a source or operation result, not its current value.
Evaluation	One forward run using one source table.
Host	Code around the graph: events, time, commits, DOM writes, and possibly training.
Weight	A numeric coefficient; distinguish compiled projection coefficients from trainable parameter cells.
Loss	A scalar objective; small loss need not imply correctness or generalization.
Gradient	Local sensitivities of a scalar objective to selected quantities.
Training	Using feedback to update selected parameters; not a synonym for evaluation.

Backend	An implementation of execution; a tensor representation does not imply a GPU.

12.4 Where to read next

These paths are relative to the repository root. They are the primary sources for this notebook, not claims of external novelty.

- `docs/tutorial.md`: cell declarations, the real switch, host timing, drawing, and comparison with plain JavaScript.
- `src/runtime/cells.ts`: `createSourceTable`, cell policies, validators, and `createEvaluator`.
- `src/runtime/numeric-ui-core.ts`: four instruction kinds, `Circuit.multiply`, `Circuit.select`, `backpropagateReference`, and `compileTensorProgram`.
- `docs/execution-model.md`: FP32 accumulation, rounding points, comparison contracts, index-gradient options, and compiled buffer lifetime.
- `template/toggle.ts` and `template/toggle-reference.ts`: the graph-based switch alongside the conventional implementation.
- `src/runtime/host-timing.ts`: spring matrices and the host's event-time advancement.
- `src/handwiredos/README.md`: the desktop's graph/host boundary.
- `src/learned-gallery/README.md` and `trainable-geometry.ts`: the actual fitted layout network, objective, optimizer, and training bookkeeping.
- `docs/expressions.md` and `docs/patterns.md`: how to express larger calculations once these basics are comfortable.

For external background and prior-art searches, start with automatic differentiation, differentiable programming, sparse dataflow compilation, inverse graphics/layout, and surrogate gradients for discrete decisions. Frameworks such as JAX and PyTorch are important baselines, not evidence that Handwired invented these ideas.

> **The destination**
>
> You understand the fundamentals when you can point to each number and say: who supplies it, which operations read it, what its legal values are, whether it persists, and whether an error has a meaningful derivative with respect to it. From there, a worthwhile next step is a falsifiable experiment—not a bigger promise.